

## Festival

Nayra è a un festival e sta giocando a un gioco il cui primo premio è un viaggio a Laguna Colorada. Il gioco consiste nell'utilizzare i gettoni per acquistare dei coupon. L'acquisto di un coupon può moltiplicare i gettoni rimanenti. L'obiettivo è ottenere il **maggior numero possibile di coupon**.

Nayra inizia il gioco con  $A$  gettoni. Ci sono  $N$  coupon, numerati da 0 a  $N - 1$ . Nayra deve pagare  $P[i]$  gettoni ( $0 \leq i < N$ ) per acquistare il coupon  $i$  (e deve avere almeno  $P[i]$  gettoni prima dell'acquisto). Può acquistare ogni coupon al massimo una volta.

Inoltre, ogni coupon  $i$  ( $0 \leq i < N$ ) ha un **tipo**, indicato con  $T[i]$ , che è un numero intero **tra 1 e 4, inclusi**. Dopo che Nayra ha acquistato il coupon  $i$ , il numero rimanente di gettoni che possiede viene moltiplicato per  $T[i]$ . Formalmente, se a un certo punto del gioco ha  $X$  gettoni, e acquista il coupon  $i$  (che richiede  $X \geq P[i]$ ), allora Nayra avrà  $(X - P[i]) \cdot T[i]$  gettoni dopo l'acquisto.

Il vostro compito è determinare quali coupon Nayra deve acquistare e in quale ordine, per massimizzare il numero totale di **coupon** che ha alla fine. Se c'è più di una sequenza di acquisti che raggiunge questo obiettivo, potete segnalarne una qualsiasi.

## Dettagli di implementazione

È necessario implementare la seguente funzione.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- $A$ : il numero iniziale di gettoni di cui dispone Nayra.
- $P$ : un array di lunghezza  $N$  che specifica i prezzi dei coupon.
- $T$ : un array di lunghezza  $N$  che specifica i tipi di coupon.
- Questa funzione viene richiamata esattamente una volta per ogni caso di test.

La funzione deve restituire un array di interi  $R$ , che specifica gli acquisti di Nayra come:

- La lunghezza di  $R$  deve essere il numero massimo di coupon che Nayra può acquistare.
- Gli elementi dell'array sono gli indici dei coupon da acquistare, in ordine cronologico. In altre parole, la donna acquisterà prima il coupon  $R[0]$ , poi il coupon  $R[1]$  e così via.
- Tutti gli elementi di  $R$  devono essere diversi tra loro.

Se non è possibile acquistare alcun coupon,  $R$  deve essere un array vuoto.

## Assunzioni

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$  per ogni  $i$  tale che  $0 \leq i < N$ .
- $1 \leq T[i] \leq 4$  per ogni  $i$  tale che  $0 \leq i < N$ .

## Subtask

| Subtask | Punteggio | Assunzioni aggiuntive                                                |
|---------|-----------|----------------------------------------------------------------------|
| 1       | 5         | $T[i] = 1$ per ogni $i$ tale che $0 \leq i < N$ .                    |
| 2       | 7         | $N \leq 3000$ ; $T[i] \leq 2$ per ogni $i$ tale che $0 \leq i < N$ . |
| 3       | 12        | $T[i] \leq 2$ per ogni $i$ tale che $0 \leq i < N$ .                 |
| 4       | 15        | $N \leq 70$                                                          |
| 5       | 27        | Nayra può acquistare tutti gli $N$ coupon (in un certo ordine).      |
| 6       | 16        | $(A - P[i]) \cdot T[i] < A$ per ogni $i$ tale che $0 \leq i < N$ .   |
| 7       | 18        | Nessuna assunzione aggiuntiva.                                       |

## Esempi

### Esempio 1

Si consideri la seguente chiamata.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra dispone inizialmente di  $A = 13$  gettoni. Può acquistare 3 coupon in quest'ordine:

| Coupon acquistato | Prezzo del coupon | Tipo di coupon | Numero di gettoni dopo l'acquisto |
|-------------------|-------------------|----------------|-----------------------------------|
| 2                 | 8                 | 3              | $(13 - 8) \cdot 3 = 15$           |
| 3                 | 14                | 4              | $(15 - 14) \cdot 4 = 4$           |
| 0                 | 4                 | 1              | $(4 - 4) \cdot 1 = 0$             |

In questo esempio, non è possibile per Nayra acquistare più di 3 coupon, e la sequenza di acquisti sopra descritta è l'unico modo in cui può acquistare 3 coupon. Pertanto, la funzione deve restituire  $[2, 3, 0]$ .

## Esempio 2

Consideriamo la seguente chiamata.

```
max_coupons(9, [6, 5], [2, 3])
```

In questo esempio, Nayra può acquistare entrambi i coupon in qualsiasi ordine. Pertanto, la funzione dovrebbe restituire o  $[0, 1]$  o  $[1, 0]$ .

## Esempio 3

Consideriamo la seguente chiamata.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

In questo esempio, Nayra ha un solo gettone, che non è sufficiente per acquistare alcun coupon. Pertanto, la funzione dovrebbe restituire  $[]$  (un array vuoto).

## Grader di esempio

Formato di input:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato di output:

```
S
R[0] R[1] ... R[S-1]
```

Dove  $S$  è la lunghezza dell'array  $R$  restituito da `max_coupons`.